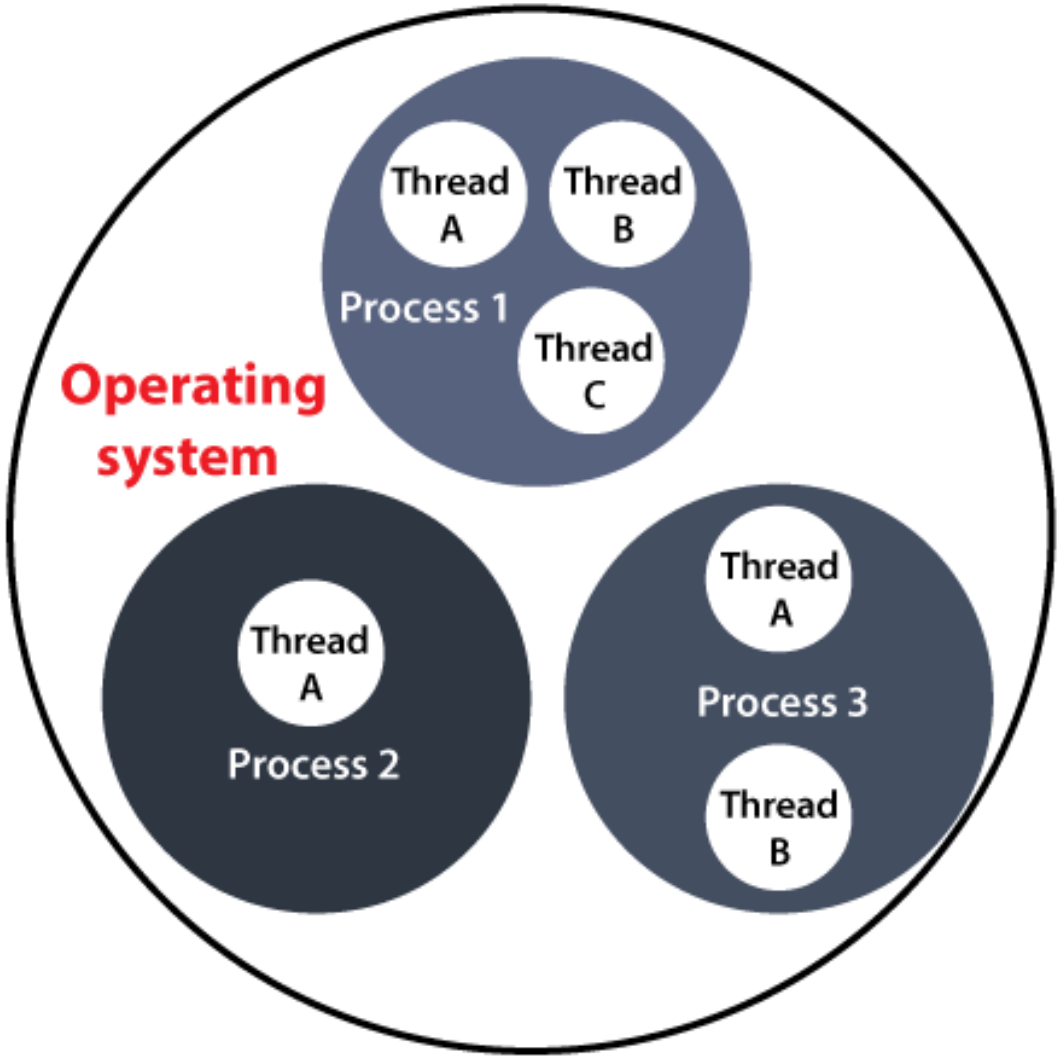
Java - MultiThreading

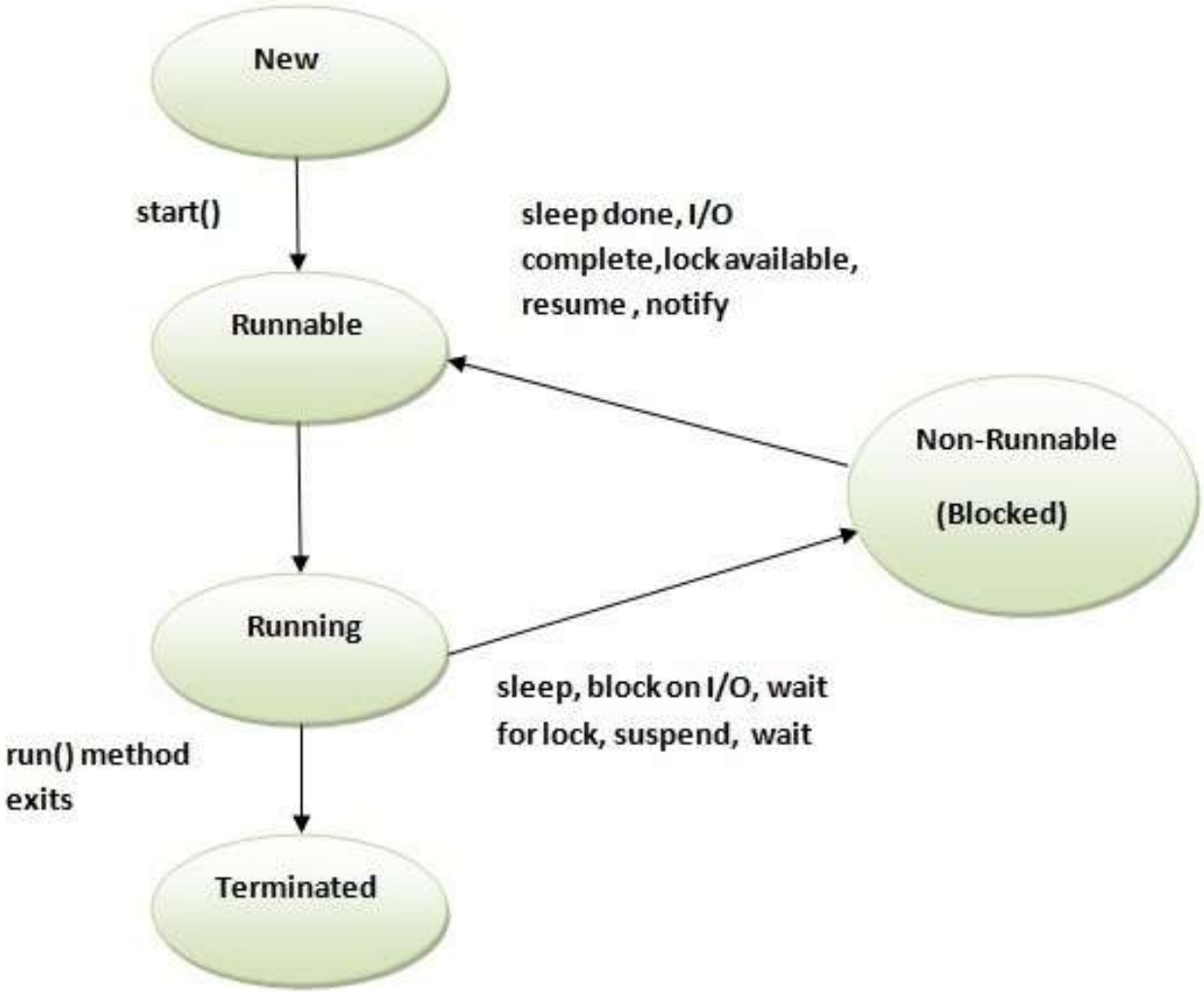
- A **Thread** is a very light-weighted process, or we can say the smallest part of the process that allows a program to operate more efficiently by running multiple tasks simultaneously.
- In order to perform complicated tasks in the background, we used the **Thread concept in Java**. All the tasks are executed without affecting the main program. In a program or process, all the threads have their own separate path for execution, so each thread of a process is independent.

- Another benefit of using **thread** is that if a thread gets an exception or an error at the time of its execution, it doesn't affect the execution of the other threads. All the threads share a common memory and have their own stack, local variables and program counter. When multiple threads are executed in parallel at the same time, this process is known as **Multithreading**.


Thread
A
Thread
B
Process 1
Thread
C
Operating
system
Thread
A
Process 2
Thread
A
Process 3
Thread
B

Life cycle of a Thread (Thread States)

- The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:
- New
- Runnable
- Running
- Non-Runnable (Blocked)
- Terminated


New
start()
Runnable
sleep done, I/O
complete,lock available,
resume , notify
Non-Runnable
(Blocked)
Running
sleep, block on I/O, wait
for lock, suspend, wait
run() method
exits
Terminated

- New Thread: When a new thread is created, it is in the new state. The thread has not yet started to run when the thread is in this state. When a thread lies in the new state, its code is yet to be run and hasn’t started to execute.
- Runnable State: A thread that is ready to run is moved to a runnable state. In this state, a thread might be ready to run at any instant of time. It is the responsibility of the thread scheduler to give the thread, time to run.
- A multi-threaded program allocates a fixed amount of time to each individual thread. Each and every thread runs for a short while and then pauses and relinquishes the CPU to another thread so that other threads can get a chance to run. When this happens, all such threads that are ready to run, waiting for the CPU lie in a runnable state.
- Blocked: The thread will be in blocked state when it is trying to acquire a lock but currently the lock is acquired by the other thread. The thread will move from the blocked state to runnable state when it acquires the lock.

- Waiting state: The thread will be in waiting state when it calls wait() method or join() method. It will move to the runnable state when other thread will notify or that thread will be terminated.
- Timed Waiting: A thread lies in a timed waiting state when it calls a method with a time-out parameter. A thread lies in this state until the timeout is completed or until a notification is received. For example, when a thread calls sleep or a conditional wait, it is moved to a timed waiting state.
- Terminated State: A thread terminates because of either of the following reasons:

✓ Because it exits normally. This happens when the code of the thread has been entirely executed by the program.

✓ Because there occurred some unusual erroneous event, like an unhandled exception.

THE THREAD CLASS AND THE RUNNABLE INTERFACE

- To create a new thread, your program will either **extend Thread** or **implement the Runnable interface**.

- **Commonly used Constructors of Thread class:**
- Thread()
- Thread(String name)
- Thread(Runnable r)
- Thread(Runnable r, String name)

- **Commonly used methods of Thread class:**
- **public void run():** is used to perform action for a thread.
- **public void start():** starts the execution of the thread.JVM calls the run() method on the thread.
- **public void sleep(long milliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
- **public void join():** The *join*() *method* is used to hold the execution of currently running thread until the specified thread is dead(finished execution). The *join()* method allows one thread to wait for the completion of another.
- **public void join(long milliseconds):**used to wait for the thread on which it's called to be dead or wait for specified milliseconds.

- **public int getPriority():** returns the priority of the thread.
- **public int setPriority(int priority):** changes the priority of the thread.
- **public String getName():** returns the name of the thread.
- **public void setName(String name):** changes the name of the thread.
- **public Thread currentThread():** returns the reference of currently executing thread.

- **public int getId():** returns the id of the thread.
- **public boolean isAlive():** tests if the thread is alive.
- **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads to execute.

Creating thread by extending Thread Class

- **Extending the java.lang.Thread Class**
- Extend the Thread class in our class
- Then creating objects of class acts as threads
- Through the methods in this class, threads can be started, stopped, interrupted, named, prioritized, and queried regarding their current state

- **Overriding the run() Method**

After extending Thread, the next step is to override the run() method

public void run() { }

When a new thread is started, the entry point into the program is the run() method.

The first statement in run() will be the first statement executed by the new thread.

The new thread is considered to be alive from just before run() is called until just after run() returns, at which time the thread dies.

After a thread has died, it cannot be restarted.

- **Invoke the start() method**
- After the new thread is created, it will not start running until you call its start( ) method
- The start() method causes and not actually starts execution.
- It schedules the thread and when CPU scheduler picks this thread for execution then JVM calls the run() method to actually start execution.
- Thread can call start() method only once in a program.

```
class NewThread extends Thread{
public void run(){
for(int i=0;i<5;i++){
System.out.println("Value of i "+i);
}
}
}
class ExThread{
public static void main(String args[]){
NewThread t=new NewThread();
System.out.println(t.getName());
t.start();
}
}
```

Implementing Runnable interface

- Create a class that implements the interface and override the run() method
- Create an object of the class
- Create Thread Object
- Start execution of the Thread using start()

```
class NewThread implements Runnable{
public void run(){
for(int i=0;i<5;i++){
System.out.println("Value of i "+i);
}
}
}
class ImplementThread{
public static void main(String args[]){
NewThread nt=new NewThread();
Thread t=new Thread(nt);
t.start();
}
}
```

Thread Class vs. Runnable Interface

- Implementing the Runnable Interface defines the unit of work that will be executed in a thread
- When extending the Thread class, the derived class cannot extend any other base classes because java only allows single inheritance
- By implementing Runnable Interface, the class can still extend other base classes if necessary.
- By extending Thread, each of your threads has a unique object associated with it, whereas implementing Runnable, many threads can share the same runnable instance or it shares the same object to multiple threads.

- **Thread Scheduler in Java**
- **Thread scheduler** in java is the part of the JVM that decides which thread should run.
- There is no guarantee that which runnable thread will be chosen to run by the thread scheduler.
- The thread scheduler mainly uses preemptive or time slicing scheduling to schedule the threads.

Sleep method in java

```
class NewThread extends Thread{
public void run(){
for(int i=0;i<5;i++){
try{Thread.sleep(500);}catch(Exception e){}
System.out.println("Value of i "+i);
}}}
class ExThread{
public static void main(String args[]){
NewThread t1=new NewThread();
NewThread t2=new NewThread();
t1.start();
t2.start();}}
```

- o/p
- Value of i 0
- Value of i 0
- Value of i 1
- Value of i 1
- Value of i 2
- Value of i 2
- Value of i 3
- Value of i 3
- Value of i 4
- Value of i 4

Priority of a Thread (Thread Priority)

- Each thread have a priority.
- Priorities are represented by a number between 1 and 10.
- In most cases, thread scheduler schedules the threads according to their priority (known as pre-emptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

- public static int MIN_PRIORITY
- public static int NORM_PRIORITY
- public static int MAX_PRIORITY

✓ Default priority of a thread is 5 (NORM_PRIORITY).

✓ The value of MIN_PRIORITY is 1

✓ The value of MAX_PRIORITY is 10.

```
class One extends Thread {
 public void run()   {
       System.out.println(getName()+" "+getPriority());
    } }
class MainThread {
 public static void main(String args[]) {
 One t1=new One();
 One t2=new One();
 One t3=new One();
t1.setPriority(Thread.NORM_PRIORITY);
t2.setPriority(Thread.MIN_PRIORITY);
t3.setPriority(Thread.MAX_PRIORITY);
  t1.start();
  t2.start();
  t3.start();
  } }
```

- To implement a multithreaded application that has 3 threads. First thread generates random integer for every second and if the value is even, second thread computes the square of number and prints. If the value is odd, third thread will print the value of cube of number

```
class Even implements Runnable{
int x;
Even(int x){
this.x=x;
}
public void run(){
System.out.println("Even Thread is "+x+" and square is "+x*x);
}
}
```

```
class Odd implements Runnable{
int x;
Odd(int x){
this.x=x;
}
public void run(){
System.out.println("Odd Thread is "+x+" and cube is "+x*x*x);
}
}
```

```
class MultiThread extends Thread{
String tName;
Random r;
Thread t1,t2;
MultiThread(String s){
tName=s;
}
public void run(){
int num=0;
r=new Random();
try{
for(int i=0;i<10;i++){
num=r.nextInt(100);
System.out.println("The random number generated is "+num);
```

```
if(num%2==0){
t1=new Thread(new Even(num));
t1.start();
}
else{
t2=new Thread(new Odd(num));
t2.start();
}
Thread.sleep(1000);
}
}catch(Exception e){System.out.println(e);}
}
}
```

```
public class MultiThreadDemo{
public static void main(String args[]){
MultiThread mt=new MultiThread("One");
mt.start();
}
}
```

Synchronization in Java

- Synchronization in Java is the capability to control the access of multiple threads to any shared resource.
- Java Synchronization is better option where we want to allow *only one thread to access the shared resource*.

  Why use Synchronization?
- The synchronization is mainly used to

✓ To prevent thread interference.

✓ To prevent consistency problem.

- Thread Synchronization
- There are two types of thread synchronization mutual exclusive and inter-thread communication.

1. **Mutual exclusive**: Keeps threads from interfering with each other while sharing resources.
2. **Inter-thread communication**: Allows one thread to pause and another to enter or lock the same critical section

- Mutual Exclusive

✓ Synchronized method.
✓ Synchronized block.
✓ Static synchronization.

- **Concept of Lock in Java**
- Synchronization is built around an internal entity known as the lock or monitor.
- Every object has a lock associated with it.
- By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them.

- Java Synchronized Method
- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

- Synchronized Block in Java
- Synchronized block can be used to perform synchronization on any specific resource of the method.
- Suppose we have 50 lines of code in our method, but we want to synchronize only 5 lines, in such cases, we can use synchronized block.
- If we put all the codes of the method in the synchronized block, it will work same as the synchronized method.

- Static Synchronization
- If you make any static method as synchronized, the lock will be on the class not on object.